UNIVERSITE PROTESTANTE AU CONGO
FACULTE DES SCIENCES INFORMATIQUES

TRAVAUX DIRIGES

Programmation Web – PHP

Sujet : Systeme de Gestion des Auditoires et des Horaires

Annee : **2025–2026**

Promotions : L2
Langage : PHP **Persistance :** Fichiers JSON ou TXT

1. Mise en Situation et Contexte

La **Faculte des Sciences Informatiques** voit ses effectifs croitre d'annee en annee, rendant la gestion manuelle des salles et des horaires de plus en plus laborieuse et source d'erreurs repetees. L'**apparitorat facultaire** — le service administratif responsable de la logistique pedagogique — se retrouve regulierement confronte a des situations problematiques : deux promotions affectees au meme auditoire sur le meme creneau, des salles sous-exploitees pendant que d'autres debordent, ou encore des cours d'option planifies sans respecter les contraintes de capacite des espaces disponibles.

Face a cette situation, l'apparitorat souhaite se doter d'un **Systeme de Gestion des Auditoires** (`SGA`), une application web developpee en **PHP procedural** capable de :

— repertorier les 6 salles disponibles avec leurs capacites respectives ;

— enregistrer les 4 promotions (L1, L2, L3, L4), leurs effectifs et leurs grilles de cours ;

— distinguer les **cours de tronc commun** (suivis par toute une promotion) des **cours d'option** specifiques aux L3 et L4 ;

— proposer automatiquement une **repartition hebdomadaire** des creneaux horaires, en affectant chaque cours a une salle adaptee, sans collision ni depassement de capacite ;

— **sauvegarder** toutes les donnees et le planning genere dans des fichiers (`.json` ou `.txt`), et permettre leur **rechargement** ulterieur.

Vous etes mandates en tant qu'etudiants developpeurs pour concevoir et implementer ce systeme **uniquement en PHP procedural**, sans recours a aucune base de donnees ni a la programmation orientee objet.

2. Description de l'Environnement

2.1. Les Salles Disponibles

La faculte dispose de **6 espaces pedagogiques** aux capacites differentes. Ces capacites constituent des **donnees d'entree** fournies au systeme via un fichier ; elles peuvent evoluer d'une annee a l'autre.

Identifiant	Designation	Capacite (a fournir)
AUD-L1	Auditoire principal – Licence 1	*a saisir*
AUD-L2	Auditoire principal – Licence 2	*a saisir*
AUD-L3	Auditoire principal – Licence 3	*a saisir*
AUD-L4	Auditoire principal – Licence 4	*a saisir*
SALLE-MACH	Salle de machines (TP informatiques)	*a saisir*
SALLE-MGT	Salle de management & marketing	*a saisir*

⚠ **Contrainte fondamentale :** Une salle ne peut etre attribuee a un groupe que si l'effectif de ce groupe est *strictement inferieur ou egal* a la capacite de la salle. Toute affectation violant cette regle doit etre rejetee par le systeme.

2.2. Les Promotions et Leurs Effectifs

La faculte compte **4 promotions** : L1, L2, L3 et L4. L'effectif de chaque promotion est une **donnee d'entree** lue depuis un fichier. Les promotions L3 et L4 comportent en outre des **groupes d'option** dont l'effectif est precise separement.

2.3. Structure des Cours

Les cours sont repartis en deux categories :

1. **Cours de tronc commun** — suivis par l'ensemble des etudiants d'une promotion. Exemples : Circuits Logique, Reseaux, Systemes d'exploitation, Bases de donnees, Freedom.

2. **Cours d'option** (*L3 et L4 uniquement*) — suivis par un sous-groupe d'etudiants ayant choisi une filiere particuliere. Exemples de filieres : Sécurité Informatique, Ingénièrie Logiciel, Robotique, Data Sciences. Systemes Embarques.

La liste complete des cours, leur volume horaire hebdomadaire et leur rattachement (promotion ou option) sont egalement des **donnees d'entree** stockees dans des fichiers.

> ℹ **Plage horaire de reference :** La semaine pedagogique s'etend du **lundi au vendredi**, de **8h00 a 17h00**, decoupee en creneaux de **4 heures** (avec pauses a definir). Un meme auditoire ne peut pas accueillir deux groupes sur le meme creneau.

2.4. Format des Fichiers de Donnees

Toutes les donnees du systeme sont stockees et relues sous forme de fichiers texte structures. Le tableau ci-dessous decrit les fichiers attendus :

`Fichier`	**Contenu**	**Format**
`salles.json`	Identifiant, designation, capacite de chaque salle	JSON ou TXT
`promotions.json`	Identifiant promotion, libelle, effectif total	json ou TXT
`cours.json`	Identifiant cours, intitule, volume horaire, promotion/option	json ou TXT
`options.json`	Identifiant option, libelle, promotion parente, effectif	JSON ou TXT
`planning.json`	Planning genere : creneau, salle, cours, groupe	JSON ou TXT

3. Travail Demande

; Partie I — Analyse et Conception

Q1. Analyse des besoins : A partir de l'enonce, dressez la liste complete des fonctionnalites attendues du systeme `SGA`. Classez-les en fonctionnalites *principales* et fonctionnalites *secondaires*.

Q2. Identification des donnees : Recensez toutes les donnees que le systeme doit manipuler (salles, promotions, cours, options, creneaux, affectations). Pour chaque type de donnee, precisez les champs necessaires et leur type (entier, chaine de caracteres, booleen, etc.).

Q3. Conception des fichiers : Definissez precisement la structure de chacun des fichiers listes en section 2.4. Precisez le separateur utilise, l'ordre des champs et fournissez un exemple de contenu complet pour chaque fichier (au moins 3 lignes par fichier).

Q4. Identification des contraintes : Listez toutes les regles metier que le systeme doit respecter lors de la generation du planning. Pour chacune, indiquez la condition a verifier et le comportement attendu en cas de violation.

Partie II — Implementation PHP Procedurale

Q5. Lecture des fichiers de donnees : Ecrivez les fonctions PHP necessaires pour lire et charger en memoire (tableaux associatifs) le contenu de chacun des fichiers de donnees. Votre code doit gerer les cas d'erreur : fichier introuvable, ligne malformee, valeur manquante.

Signatures attendues :

```php
function charger_salles($chemin_fichier) { ... }
function charger_promotions($chemin_fichier) { ... }
function charger_cours($chemin_fichier) { ... }
function charger_options($chemin_fichier) { ... }
```

Listing 1 – Fonctions de lecture des fichiers

Q6. Verification des contraintes : Ecrivez les fonctions de validation permettant de tester, avant toute affectation, que les regles metier sont bien respectees.

Signatures attendues :

```php
function salle_disponible($planning, $id_salle, $creneau)
    { ... }
function capacite_suffisante($salles, $id_salle,
   $effectif) { ... }
function creneau_libre_groupe($planning, $id_groupe,
   $creneau) { ... }
```

Listing 2 – Fonctions de verification des contraintes

Q7. Generation du planning : Ecrivez la fonction principale qui, a partir des donnees chargees, produit une proposition de planning hebdomadaire sans conflit. Vous decrirez et justifierez la strategie d'affectation retenue.

Signature attendue :

```php
function generer_planning($salles, $promotions,
                          $cours, $options,
                          $creneaux_disponibles) { ... }
```

Listing 3 – Fonction de generation du planning

Q8. Sauvegarde du planning dans un fichier : Ecrivez une fonction qui prend en parametre le planning genere et le sauvegarde dans le fichier `planning.txt` selon un format structure et lisible. Chaque ligne devra comporter : le jour, l'heure de debut, l'heure de fin, l'identifiant de la salle, le code du cours et l'identifiant du groupe.

Signature attendue :

```php
function sauvegarder_planning($planning, $chemin_fichier)
    { ... }
```

Listing 4 – Fonction de sauvegarde

Q5. Rechargement et affichage du planning : Ecrivez une fonction qui lit le fichier `planning.json` sauvegarde et affiche son contenu sous forme de tableau HTML hebdomadaire (jours en colonnes, creneaux en lignes), en indiquant pour chaque case la salle, le cours et le groupe concerne.

Signatures attendues :

```php
function charger_planning($chemin_fichier) { ... }
function afficher_planning_html($planning) { ... }
```

Listing 5 – Fonctions de rechargement et d'affichage

Q6. Script principal : Ecrivez le script `index.php` qui orchestre l'ensemble du systeme : chargement des fichiers, generation du planning, sauvegarde et affichage. Ce script doit afficher des messages clairs en cas de succes ou d'erreur a chaque etape du traitement.

★ Partie III — Extensions

B1. Detection de conflits : Ecrivez une fonction qui analyse le fichier `planning.txt` deja sauvegarde et detecte automatiquement tous les conflits presents (meme salle sur le meme creneau, meme groupe sur deux creneaux simultanes). Les conflits detectes doivent etre listes de facon explicite a l'ecran.

B2. Rapport d'occupation des salles : Ajoutez une fonctionnalite qui lit le planning sauvegarde et genere un fichier `rapport_occupation.txt` indiquant, pour chaque salle, le nombre de creneaux occupes, le nombre de creneaux libres et le taux d'occupation en pourcentage sur la semaine.

B3. Mise a jour manuelle du planning : Proposez des fonctions permettant de modifier manuellement une affectation dans le fichier `planning.txt` (changer de salle ou de creneau pour un cours donne) avec verification immediate des contraintes avant ecriture.

B4. Formulaire HTML/PHP de saisie : Proposez des formulaires HTML permettant a l'apparitorat de saisir ou modifier les donnees de configuration (salles, promotions, cours) directement depuis le navigateur, avec enregistrement automatique dans les fichiers `.csv` correspondants.

4. Modalites de Rendu

Format	Archive `.zip` nommee `NOM_Prenom_SGA.zip`
Contenu	Scripts PHP, fichiers JSON/TXT de donnees, rapport PDF (max 8 pages)
Rapport	Analyse des besoins, structure des fichiers, choix de conception, captures d'ecran des resultats
Delai	A preciser par l'enseignant

5. Conseils Methodologiques

— **Ne sautez pas l'analyse :** definissez la structure de vos fichiers avant d'ecrire la moindre ligne de PHP. Un mauvais format de fichier en debut de projet entraine des corrections couteuses.

— **Une fonction = une responsabilite :** chaque fonction ne doit realiser qu'une seule action (lire un fichier, verifier une contrainte, ecrire un resultat). Evitez les fonctions de plusieurs dizaines de lignes.

— **Gerez les erreurs fichier :** utilisez `file_exists()`, `fopen()` et `fclose()` correctement, et affichez des messages explicites si un fichier est absent ou corrompu.

— **Testez chaque fonction separement :** avant d'assembler le systeme complet, verifiez que chaque fonction produit le resultat attendu sur des donnees simples.

— **Commentez votre code :** chaque fonction doit etre precedee d'un commentaire indiquant son role, ses parametres et sa valeur de retour.

— **Sauvegardez vos fichiers de donnees :** conservez des copies de vos fichiers JSON avant chaque test pour pouvoir revenir en arriere en cas d'ecriture erronee.

Bon travail a toutes et a tous !